Cosmos EVM JSON-RPC 서버 및 이더리움 호환성

1. JSON-RPC 서버 개요

1.1 목적

Cosmos EVM의 JSON-RPC 서버는 기존 이더리움 도구들(MetaMask, Remix, Hardhat 등)과의 완전한 호환성을 제공합니다.

1.2 지원 네임스페이스

- **eth**: 이더리움 표준 메서드
- **net**: 네트워크 정보
- **web3**: Web3 표준 메서드
- **txpool**: 트랜잭션 풀 관리
- **debug**: 디버깅 기능
- **miner**: 마이닝 관련 (제한적)

1.3 서버 구조

```
type JSONRPCServer struct {
// HTTP 서버
httpServer *http.Server
// WebSocket 서버
```

```
wsServer *websocket.Server

// 백엔드 인터페이스

backend Backend

// 설정

config *Config

}
```

2. 백엔드 구현

2.1 Backend 인터페이스

```
type Backend interface {

// 블록체인 정보

ChainID() *big.Int

CurrentHeader() *ethtypes.Header

HeaderByNumber(blockNum rpc.BlockNumber) (*ethtypes.Header, error)

BlockByNumber(blockNum rpc.BlockNumber) (*ethtypes.Block, error)

// 계정 정보

GetBalance(address common.Address, blockNum rpc.BlockNumber) (*big.Int, error)

GetCode(address common.Address, blockNum rpc.BlockNumber) ([]byte, error)

GetNonce(address common.Address, blockNum rpc.BlockNumber) (uint64, error)

// 트랜잭션 처리

SendTransaction(args TransactionArgs) (common.Hash, error)
```

```
EstimateGas(args TransactionArgs, blockNrOrHash *rpc.BlockNumberOrHash) (hexutil

// 상태 조회

GetStorageAt(address common.Address, key  string, blockNum rpc.BlockNumber) (hex

Call(args TransactionArgs, blockNrOrHash *rpc.BlockNumberOrHash) (hexutil.Bytes,

}
```

2.2 Cosmos EVM Backend 구현

```
type CosmosEVMBackend struct {

// Cosmos SDK 컨텍스트

clientCtx client.Context

// 키퍼들

evmKeeper evmkeeper.Keeper

feeMarketKeeper feemarketkeeper.Keeper

// 쿼리 클라이언트

queryClient *rpc.QueryClient

}


func (b *CosmosEVMBackend) GetBalance(address common.Address, blockNum rpc.Block

// Cosmos 상태에서 잔액 조회

ctx := b.getContextForBlockNumber(blockNum)

// EVM 계정 조회

account := b.evmKeeper.GetAccount(ctx, address)
```

```
if  account == nil {
return  big.NewInt(0), nil
}
return  account.Balance, nil
}
```

3. 주요 JSON-RPC 메서드 구현

3.1 eth_getBalance

```
func (api *PublicEthereumAPI) GetBalance(address common.Address, blockNrOrHash r
blockNum, err := api.backend.BlockNumberFromTendermint(blockNrOrHash)
if  err != nil {
return  nil, err
}
balance, err := api.backend.GetBalance(address, blockNum)
if  err != nil {
return  nil, err
}
return (*hexutil.Big)(balance), nil
}
```

3.2 eth_sendTransaction

```
func (api *PublicEthereumAPI) SendTransaction(args TransactionArgs) (common.Hash

// 트랜잭션 파라미터 검증

if  err := args.setDefaults(api.backend.ChainConfig().ChainID);  err != nil {

return common.Hash{}, err

}

// 이더리움 트랜잭션 생성

tx := args.toTransaction()

// Cosmos 메시지로 변환

msg, err := tx.AsMessage(types.LatestSigner(api.backend.ChainConfig()), nil)

if  err != nil {

return common.Hash{}, err

}

// 트랜잭션 브로드캐스트

return  api.backend.SendTransaction(msg)

}
```

3.3 eth_call

```
func (api *PublicEthereumAPI) Call(args TransactionArgs, blockNrOrHash *rpc.Bloc

// 블록 번호 결정

blockNum := rpc.LatestBlockNumber
```

```
if  blockNrOrHash != nil {
blockNum = blockNrOrHash.BlockNumber
}
// EVM 호출 실행
result, err := api.backend.Call(args, blockNum, overrides)
if  err != nil {
return  nil, err
}
return  result, nil
}
```

3.4 eth_estimateGas

```
func (api *PublicEthereumAPI) EstimateGas(args TransactionArgs, blockNrOrHash *r
// 가스 추정을 위한 시뮬레이션 실행
estimate, err := api.backend.EstimateGas(args, blockNrOrHash)
if  err != nil {
return  0, err
}
return  estimate, nil
}
```

4. 블록 및 트랜잭션 변환

4.1 Tendermint 블록 → 이더리움 블록

```
func (b *CosmosEVMBackend) BlockByNumber(blockNum rpc.BlockNumber) (*ethtypes.Bl

// Tendermint 블록 조회

tmBlock, err := b.getTendermintBlock(blockNum)

if  err != nil {

return  nil, err

}

// 이더리움 헤더 생성

header := &ethtypes.Header{

Number: big.NewInt(tmBlock.Height),

Time: uint64(tmBlock.Time.Unix()),

ParentHash: common.BytesToHash(tmBlock.LastBlockID.Hash),

Root: common.BytesToHash(tmBlock.AppHash),

GasLimit: b.getBlockGasLimit(),

GasUsed: b.getBlockGasUsed(tmBlock),

}

// 트랜잭션 변환

var  ethTxs []*ethtypes.Transaction

for  _, tx := range  tmBlock.Txs {

if  ethTx := b.convertToEthTx(tx);  ethTx != nil {
```

```
ethTxs = append(ethTxs, ethTx)
}
}
return ethtypes.NewBlock(header, ethTxs, nil, nil, trie.NewStackTrie(nil)), nil
}
```

4.2 Cosmos 트랜잭션 → 이더리움 트랜잭션

```
func (b *CosmosEVMBackend) convertToEthTx(cosmosTx []byte) *ethtypes.Transaction
// Cosmos 트랜잭션 디코딩
tx, err := b.clientCtx.TxConfig.TxDecoder()(cosmosTx)
if err != nil {
return nil
}
// EVM 메시지 추출
for _, msg := range tx.GetMsgs() {
if ethMsg, ok := msg.(*evmtypes.MsgEthereumTx); ok {
// 이더리움 트랜잭션으로 변환
ethTx := ethMsg.AsTransaction()
return ethTx
}
}
return nil
```

```
}
```

5. WebSocket 지원

5.1 WebSocket 서버 설정

```
type WebSocketServer struct {
upgrader websocket.Upgrader
// 구독 관리
subscriptions  map[string]*Subscription
// 이벤트 필터
filters  map[string]*Filter
}

func (ws *WebSocketServer) HandleConnection(w http.ResponseWriter, r *http.Reque
conn, err := ws.upgrader.Upgrade(w, r, nil)
if  err != nil {
return
}
defer  conn.Close()
// 메시지 처리 루프
for {
var  req JSONRPCRequest
```

```
if  err := conn.ReadJSON(&req);  err != nil {
break
}
response := ws.handleRequest(req)
conn.WriteJSON(response)
}
}
```

5.2 이벤트 구독

```
func (api *PublicFilterAPI) NewHeads(ctx context.Context) (*rpc.Subscription, er
notifier, supported := rpc.NotifierFromContext(ctx)
if !supported {
return  nil, rpc.ErrNotificationsUnsupported
}
rpcSub := notifier.CreateSubscription()
go  func() {
// 새 블록 이벤트 구독
blockCh := make(chan *ethtypes.Header)
api.backend.SubscribeNewHeads(blockCh)
for {
select {
case  header := <-blockCh:
```

```
notifier.Notify(rpcSub.ID, header)
case <-rpcSub.Err():
return
}
}
}()
return  rpcSub, nil
}
```

6. 로그 및 이벤트 필터링

6.1 로그 필터 구현

```
type FilterQuery struct {
BlockHash *common.Hash
FromBlock *big.Int
ToBlock *big.Int
Addresses []common.Address
Topics [][]common.Hash
}

func (api *PublicFilterAPI) GetLogs(ctx context.Context, crit FilterQuery) ([]*e
// 블록 범위 결정
```

```
fromBlock := crit.FromBlock

toBlock := crit.ToBlock

var  logs []*ethtypes.Log

// 블록별로 로그 수집

for  blockNum := fromBlock;  blockNum.Cmp(toBlock) <= 0;  blockNum.Add(blockNum,

blockLogs, err := api.backend.GetLogsByBlockNumber(blockNum)

if  err != nil {

continue

}

// 필터 적용

filteredLogs := api.filterLogs(blockLogs, crit)

logs = append(logs, filteredLogs...)

}

return  logs, nil

}
```

6.2 실시간 로그 구독

```
func (api *PublicFilterAPI) NewFilter(crit FilterQuery) (rpc.ID, error) {

// 필터 ID 생성

filterID := rpc.NewID()

// 필터 등록
```

```
filter := &Filter{
ID: filterID,
Query: crit,
Logs: make([]*ethtypes.Log, 0),
Created: time.Now(),
}
api.filters[filterID] = filter
// 백그라운드에서 로그 수집
go  api.collectLogs(filter)
return  filterID, nil
}
```

7. 디버깅 지원

7.1 트랜잭션 트레이싱

```
func (api *PrivateDebugAPI) TraceTransaction(ctx context.Context, hash common.Ha
// 트랜잭션 조회
tx, blockHash, blockNumber, index := api.backend.GetTransaction(hash)
if  tx == nil {
return  nil, fmt.Errorf("transaction %#x not found", hash)
}
// 블록 상태 복원
```

```go
statedb, err := api.backend.StateAtBlock(blockHash, blockNumber-1)
if err != nil {
return nil, err
}
// 트레이서 설정
tracer, err := api.getTracer(config)
if err != nil {
return nil, err
}
// 트랜잭션 재실행 및 트레이싱
result, err := api.backend.TraceTransaction(statedb, tx, tracer)
if err != nil {
return nil, err
}
return result, nil
}
```

7.2 블록 트레이싱

```go
func (api *PrivateDebugAPI) TraceBlock(ctx context.Context, blob hexutil.Bytes,
// 블록 디코딩
var block ethtypes.Block
if err := rlp.DecodeBytes(blob, &block); err != nil {
```

```go
return nil, err
}
// 각 트랜잭션 트레이싱
var results []*TxTraceResult
for i, tx := range block.Transactions() {
result, err := api.traceTransaction(tx, &block, i, config)
if err != nil {
results = append(results, &TxTraceResult{Error: err.Error()})
} else {
results = append(results, &TxTraceResult{Result: result})
}
}
return results, nil
}
```

8. 성능 최적화

8.1 캐싱 전략

```go
type Cache struct {
// 블록 캐시
blockCache *lru.Cache
// 트랜잭션 캐시
```

```
txCache *lru.Cache
// 상태 캐시
stateCache *lru.Cache
}

func (c *Cache) GetBlock(hash common.Hash) (*ethtypes.Block, bool) {
if  block, ok := c.blockCache.Get(hash);  ok {
return  block.(*ethtypes.Block), true
}
return  nil, false
}
```

8.2 배치 처리

```
func (api *PublicEthereumAPI) BatchCall(calls []CallArgs) ([]hexutil.Bytes, erro
results := make([]hexutil.Bytes, len(calls))
// 병렬 처리
var  wg sync.WaitGroup
for  i, call := range  calls {
wg.Add(1)
go  func(index  int, args CallArgs) {
defer  wg.Done()
result, err := api.Call(args, nil, nil)
```

```
if  err != nil {
results[index] = nil
} else {
results[index] = result
}
}(i, call)
}
wg.Wait()
return  results, nil
}
```

이 문서는 Cosmos EVM의 JSON-RPC 서버와 이더리움 호환성을 상세히 설명합니다. 다음 문서에서는 개발 및 배포 가이드를 살펴보겠습니다.